

Financial Market ETL Pipeline

Technical interview defense guide and build-it-yourself implementation plan

This document explains the ETL project decisions in enough detail to defend the design in a technical interview. It also gives a practical implementation path if you wanted to rebuild the pipeline yourself from an empty folder.

Stack: Python, requests, pandas, Pandera, SQLAlchemy, PostgreSQL, Apache Airflow, Docker Compose, pytest, python-dotenv, and Alpha Vantage.

What You Should Be Able To Say In An Interview

- The project is an idempotent daily ETL pipeline for market prices: rerunning the same date updates existing records instead of creating duplicates.
- Raw API responses are retained before transformation, which supports replay, debugging, auditability, and future schema changes.
- Data quality is enforced in two layers: Pandera validates batches before load, and PostgreSQL constraints enforce critical invariants at rest.
- Airflow owns scheduling and operational visibility, while the ETL logic lives in normal Python modules so it can be tested and run manually.
- Secrets are supplied through environment variables loaded from .env, never committed or hard-coded.

1. Requirements And How The Design Meets Them

Requirement	Design Decision	How To Defend It
Financial data from a real API	Use Alpha Vantage daily OHLCV prices for AAPL, MSFT, NVDA, AMZN, and GOOGL.	It is easy to access with a free key, returns structured JSON, and supports compact daily output. Full historical output currently requires a premium plan.
Run after the trading day ends	Airflow schedule is 0 17 * * 1-5 in America/Los_Angeles.	5 PM Pacific is after the regular U.S. market close, giving providers time to publish daily bars. Weekday scheduling is simple and transparent.
Preserve old data	Historical rows stay in market_prices. New runs upsert by ticker and trading_date.	The pipeline is append/update oriented, not destructive. It does not truncate tables during normal operation.
Prevent duplicates	PostgreSQL unique constraint on ticker + trading_date plus SQLAlchemy ON CONFLICT DO UPDATE.	Idempotency is enforced by the database, not only by application logic.
Retain raw API responses	Write raw JSON to data/raw// and store searchable metadata in raw_api_responses.	Raw retention makes failures diagnosable and transformations reproducible.
Automated quality checks	Pandera validates required columns, price relationships, volume, source, and duplicate rows.	Bad data is rejected before it becomes durable state.
Tests	Unit tests cover extraction retry behavior, transformation metrics, validation, and an optional PostgreSQL integration upsert test.	Tests focus on the riskiest behavior: API failure, correctness of calculations, and duplicate prevention.

2. Architecture Overview

The design is intentionally modular. Airflow triggers the run, but does not contain the business logic. The business logic is split into configuration, extraction, transformation, validation, loading, and orchestration modules.

```
Flow: Airflow DAG -> pipeline.py -> config.py -> extract.py -> raw JSON files -> transform.py -> validate.py -> database.py -> PostgreSQL tables.
```

- Extract: fetch daily OHLCV prices with retries and exponential backoff.
- Raw retention: write the untouched JSON response before transforming it.
- Transform: normalize the JSON into rows and compute daily return plus 7-day and 30-day moving averages.
- Validate: reject invalid or internally inconsistent batches using Pandera.
- Load: upsert into PostgreSQL and record raw response metadata.

3. Why This Tech Stack

Technology	Role	Reasonable Interview Defense
Python	Main implementation language	Strong ecosystem for APIs, data work, scheduling integrations, and tests.
requests	HTTP client	Simple, explicit, and enough for one API integration with retry logic around it.
pandas	Transformation layer	Best fit for tabular time-series shaping, sorting, percent change, and rolling averages.
Pandera	Data quality layer	Documents and enforces dataframe expectations close to the transformation code.
SQLAlchemy	Database interface	Keeps SQL operations structured while still allowing PostgreSQL-specific upsert behavior.
PostgreSQL	Warehouse table store	Reliable relational constraints, indexing, JSONB metadata, and strong upsert support.
Apache Airflow	Scheduler and orchestrator	Industry-recognized scheduler with retries, logs, DAG history, and manual reruns.
Docker Compose	Local runtime	Makes PostgreSQL and Airflow reproducible on macOS without manual service setup.
pytest	Test runner	Simple unit and integration testing with clear local commands.
python-dotenv	Configuration loading	Good local developer experience while preserving environment-variable based secrets.

A key interview point: this is not the only possible stack. For a larger production system, you might use managed Airflow, S3 or GCS for raw storage, dbt for transformations, Great Expectations for richer data tests, and a managed warehouse. This project uses a smaller stack because it is local, understandable, and complete.

4. Data Model And Idempotency

The most important database decision is the unique key on ticker and trading_date. Market prices are naturally keyed by instrument and date for this daily-grain table. The database rejects duplicate observations, and the loader uses ON CONFLICT DO UPDATE to make reruns safe.

Table	Purpose	Important Columns
raw_api_responses	Audit table for every retained raw response.	provider, ticker, requested_at, status_code, response_file_path, response_meta.
market_prices	Clean analytical table for daily prices and derived metrics.	ticker, trading_date, open, high, low, close, adjusted_close, volume, daily_return, moving averages, ingested_at. For the free daily endpoint, adjusted_close mirrors close.

- Why upsert instead of insert-only: API providers can revise historical daily bars, and reruns should correct data rather than create duplicates.
- Why keep raw JSON on disk: it is cheaper and simpler locally than storing large raw payloads directly in a relational table.
- Why also store raw metadata in PostgreSQL: files alone are hard to query; metadata makes it easy to find which response produced which load.

5. Pipeline Phases In Detail

Phase	What Happens	Failure/Quality Behavior
Configuration	Load .env values for API key, tickers, database URL, raw-data directory, log directory, and retry settings.	The pipeline fails early if the API key is missing.
Extraction	Call Alpha Vantage TIME_SERIES_DAILY for each ticker.	Timeouts, HTTP errors, rate-limit notes, and malformed API responses are retried with exponential backoff.
Raw persistence	Write each successful response as timestamped JSON under data/raw//.	Raw files support replay and diagnosis if a later phase fails.
Transformation	Convert nested JSON to a dataframe, normalize types, sort by date, and compute returns and rolling averages.	Missing daily series data raises an explicit error.
Validation	Pandera validates required columns, positive prices, volume, duplicate rows, source values, and price relationships.	Invalid batches are rejected before loading.
Loading	Insert raw response metadata and upsert cleaned rows into PostgreSQL.	Unique constraints and checks protect the database even if application validation misses something.

6. Scheduling Decision

The DAG runs at 5 PM America/Los_Angeles on weekdays. That is after the U.S. regular session closes and is a reasonable local schedule for end-of-day prices. The DAG uses catchup=False to avoid accidentally backfilling many historical Airflow runs when first enabled.

Known limitation: the weekday cron does not know about NYSE/Nasdaq holidays or early closes. The current design remains safe because upserts avoid duplicates and API failures leave old data untouched. A production upgrade would add an exchange calendar such as pandas-market-calendars and skip non-trading days intentionally.

7. Failure Handling And Observability

- Retries use exponential backoff so temporary network or provider issues do not fail the whole job immediately.
- Alpha Vantage information, note, and error-message responses are treated as failures, because they often mean rate limit, invalid symbol, or unusable payload.
- Failures are logged per ticker, and the pipeline attempts remaining tickers before raising a final error.
- Existing database records are not deleted when the API fails.
- Airflow surfaces failed task status, logs, manual reruns, and run history.

What To Say If Asked About Partial Failure

This project attempts every ticker and then fails the run if any ticker failed. That gives the operator maximum visibility while still loading successful tickers. In a stricter production pipeline, you might use separate Airflow tasks per ticker, dead-letter failed symbols, or require all-or-nothing transactions depending on the downstream contract.

8. Testing Strategy

Test Area	Current Coverage	Why It Matters
Extraction	Mocks timeout then success, and validates API error handling.	Covers the most common external dependency risks.
Transformation	Uses sample Alpha Vantage payload and checks ordering, returns, and moving average behavior.	Protects numerical correctness.
Validation	Checks valid rows pass and invalid high/low relationships fail.	Ensures data-quality rules actually enforce constraints.
Database	Optional integration test verifies upsert prevents duplicate ticker/date records.	Proves idempotency against PostgreSQL itself.

A good interview answer: unit tests isolate logic and run fast; integration tests require PostgreSQL and verify behavior that mocks cannot prove, especially database constraints and ON CONFLICT semantics.

9. Security And Configuration

- The API key is read from ALPHA_VANTAGE_API_KEY in .env and is never hard-coded.
- .env is excluded by .gitignore, while .env.example documents required settings safely.
- Docker Compose uses local development credentials only. Production would use a secrets manager and rotated database credentials.
- The pipeline writes only inside the project directory: data/raw, logs, and the configured database.
- Configuration is centralized in config.py so defaults, validation, and path handling are not scattered across the codebase.

10. How To Implement This Yourself

Use this order if you are rebuilding from scratch.

- Create the project structure: src/market_etl, tests, db/init, airflow/dags, data/raw, and logs.
- Create .env.example first. Decide the required environment variables before writing pipeline code.
- Write config.py to load .env, parse tickers, validate output-size choices, and resolve local folders.
- Write extract.py with a single function that takes explicit parameters and returns a typed payload object.
- Add retry behavior around the HTTP call. Treat rate-limit and information messages as failures.
- Save raw JSON immediately after a successful response. Use timestamped filenames per ticker.
- Write transform.py to convert one provider payload into one clean dataframe schema.
- Add derived fields only after sorting by ticker and trading_date, so percent changes and rolling averages are correct.
- Write validate.py with Pandera. Start with required columns and types, then add domain rules.
- Design the PostgreSQL tables. Add the unique ticker + trading_date constraint before writing the loader.
- Write database.py with create_tables, insert_raw_response, and upsert_market_prices.
- Write pipeline.py last. It should orchestrate modules and stay thin enough to read in one sitting.
- Add unit tests as each module is written. Add the PostgreSQL integration test once the schema exists.
- Add the Airflow DAG only after the manual command works. Airflow should call the same tested pipeline function.
- Add Docker Compose for PostgreSQL and Airflow so setup does not depend on manually installed services.
- Write README commands for setup, manual run, Airflow run, test run, and database inspection.

11. Commands You Should Understand

These are the commands worth being able to explain without memorizing every flag.

Command	What It Does
<code>cp .env.example .env</code>	Creates a local secrets/config file from the safe template.
<code>docker compose up -d postgres</code>	Starts PostgreSQL locally before manual pipeline or integration tests.
<code>PYTHONPATH=src python -m market_etl.pipeline --output-size full</code>	Runs full historical output only if the Alpha Vantage plan supports it.
<code>PYTHONPATH=src python -m market_etl.pipeline --output-size compact</code>	Runs the smaller daily update mode.
<code>pytest tests/unit</code>	Runs fast tests that do not need a database or live API.
<code>RUN_INTEGRATION_TESTS=true DATABASE_URL=... pytest tests/integration</code>	Runs the database behavior test against PostgreSQL.
<code>docker compose up --build</code>	Builds and starts Airflow plus PostgreSQL for scheduled operation.

12. Interview Questions And Strong Answers

Question	Strong Answer
Why did you choose Alpha Vantage?	It is free to start, simple to authenticate, returns daily OHLCV data in compact mode, and is good enough for a local portfolio ETL. The code isolates the provider so another API could replace it.
How do you prevent duplicates?	The database has a unique constraint on ticker and trading_date, and the loader uses PostgreSQL upsert. That makes reruns idempotent.
Why store raw responses?	Raw data supports replay, debugging, audit trails, and fixing transformation bugs without calling the provider again.
Why validate with Pandera and PostgreSQL?	Pandera catches bad batches before load. PostgreSQL constraints protect durable state. They cover different failure points.
What happens when the API is down?	The extractor retries with exponential backoff, logs failures, leaves old data untouched, and fails the Airflow task for visibility.
What would you improve for production?	Use managed secrets, object storage for raw files, an exchange calendar, provider rate-limit controls, metrics/alerts, separate tasks per ticker, and CI.

13. Production Hardening Roadmap

Area	Current Version	Production Upgrade
Raw storage	Project-local JSON files	S3, GCS, or another durable object store with lifecycle policy.
Secrets	.env for local use	Cloud secrets manager or Airflow connection backend.
Calendar	Weekday cron	Exchange calendar with holidays and early closes.
Monitoring	Local logs and Airflow task status	Metrics, alerts, SLA checks, and dashboarding.
Rate limits	Simple retry/backoff	Centralized rate-limit policy and provider usage tracking.
Data modeling	Single daily prices table	Separate dimensions, symbols metadata, corporate actions, and partitioning.
Testing	Unit plus optional integration tests	CI pipeline, contract tests with frozen API payloads, and migration tests.
Deployment	Docker Compose on macOS	Managed Airflow, managed PostgreSQL/warehouse, IaC, and release promotion.

14. Mental Model For Rebuilding It

Think in contracts. The API contract produces raw JSON. The transformation contract produces a dataframe with exact columns. The validation contract decides whether that dataframe is acceptable. The database contract decides what is unique and durable. The scheduler contract decides when the pipeline should run. If each contract is small and testable, the full ETL becomes understandable.

The most important implementation habit is to make the manual command work before Airflow. Airflow should schedule proven code, not hide untested logic inside a DAG file.

Final Checklist Before Demo Or Interview

- Be able to point to the unique constraint and explain idempotency.
- Be able to explain why raw data is retained.
- Know which tests run without services and which test needs PostgreSQL.
- Know the limitations: holidays, rate limits, local raw storage, and local credentials.
- Be ready to describe the production version without pretending this local project is already production-grade.